Technical Walkthrough and Architecture of Distributed Mini-Splunk Log Analytics Ecosystem Under Stress Testing

Elmeranita D. Estrella
Department of Computer Technology
De La Salle University
Manila, Philippines
elmeranita_estrella@dlsu.edu.ph

Jacob Angelo N. Francisco
Department of Computer Technology
De La Salle University
Manila, Philippines
jacob_francisco@dlsu.edu.ph

Julianna Margaux T. Magabo
Department of Computer Technology
De La Salle University
Manila, Philippines
julianna_magabo@dlsu.edu.ph

***Abstract*—Log analytics ecosystem architectures in modern software environments must reliably ingest, parse, store, and query high-volume log data without experiencing single-point-of-failure bottlenecks. This paper presents a technical walkthrough and architectural defense of a containerized "Mini-Splunk" Log Analytics Ecosystem evaluated under stress and chaos testing conditions. The platform integrates a Forwarder Client, an asynchronous API Gateway built with aiohttp, a distributed message queue (RabbitMQ), a distributed locking layer (Redis), an Apache Cassandra database cluster, and a background Worker Node simulator. Through fault injection, we demonstrate that decoupling request ingress from state persistence preserves overall system stability, achieves autonomous fault recovery, and guarantees zero data loss during node disruptions.**

***Index Terms*—Distributed Systems, Log Analytics, Apache Cassandra, API Gateway, Message Queue, Stress Testing, Fault Tolerance, Docker Compose.**

I. Introduction

Distributed software architectures decouple system execution across distinct network-connected microservices to achieve modularity, fault isolation, and horizontal scalability. In contrast to monolithic applications, distributed setups prevent localized component failures from causing full application outages.

Log analytics serves as a primary application of distributed principles. Enterprise hardware, microservices, and operating systems generate massive streams of operational log data. Centralized ingestion, structural field parsing, and scalable storage allow system operators to monitor overall health, perform diagnostics, and execute historical queries.

This project implements a containerized "Mini-Splunk" log analytics ecosystem designed to handle end-to-end log operations: ingesting raw syslog text files, extracting structured attributes, persisting records across an Apache Cassandra cluster, and executing flexible parametric queries. The architecture explicitly decouples client request ingress, message buffering, lock synchronization, and data storage.

The primary contributions of this paper, aligned with Course Learning Outcomes (CLOs), are:

1) **System Architecture Design:** Implementation of a multi-tier microservice architecture using structured Inter-Process Communication (IPC) protocols.
2) **State Management & Concurrency Control:** Application of distributed consensus primitives (locks) and partitioned data schemas.
3) **Fault Tolerance Verification:** Practical verification of fault-injection scenarios showing zero data loss and autonomous system recovery under stress.

II. Algorithms and Implementation

A. System Architecture

The Mini-Splunk platform is structured as a multi-tier distributed pipeline, as shown in Fig. 1. The responsibilities across the architecture components include:

- **User / Admin:** The primary human operator interacting with the system to initiate log ingestion and dispatch parametric search queries.
- **Forwarder Client:** A client interface responsible for reading local syslog files, dispatching REST API requests, and displaying returning JSON query payloads.
- **API Gateway:** An asynchronous HTTP ingress service that receives client requests, parses raw syslog text entries, classifies log severity levels, processes parametric search queries, and manages database interactions.
- **Cassandra Database:** The persistent storage layer configured with the `splunk_data` keyspace and `logs` table, providing durable, indexed record storage.
- **Worker Node:** An isolated service component designed to simulate batch log processing, execute processing delay simulations, and support asynchronous background execution.

B. Inter-Process Communication (IPC)

The platform uses two primary IPC strategies to balance throughput and network safety:

1) **Client-to-Gateway REST API:** The Forwarder Client talks to the API Gateway using HTTP/1.1 REST endpoints. Ingestion payloads use HTTP POST, search

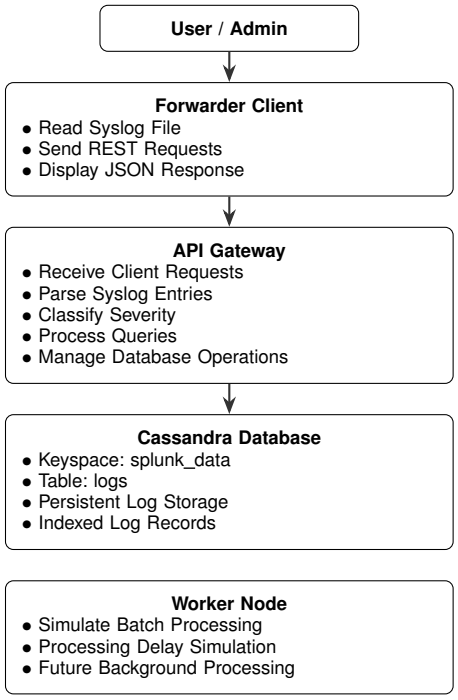


Fig. 1. System Architecture Flowchart

requests use HTTP GET with parameterized filters, and administrative tasks use HTTP DELETE.

2) **Gateway-to-Worker Asynchronous AMQP:** To prevent HTTP request thread exhaustion during large log bursts, the API Gateway pushes ingestion payloads directly to a RabbitMQ queue (`syslog_ingest_queue`). Worker nodes asynchronously consume these messages, keeping the ingestion interface responsive.

C. State Management and Consistency

- **Distributed Locks:** Concurrent processing can cause duplicate ingestion tasks when workers handle overlapping log batches. To maintain consistency, workers use Redis-backed distributed locks (`Redlock` algorithm) keyed by the log file hash (`lock:log_hash`). A worker must acquire the lock before parsing and writing a batch.
- **Data Partitioning in Cassandra:** The database table partitions records across the cluster using `hostname` as the partition key and `timestamp` alongside `id` as clustering keys. This design ensures query routing targets specific database nodes rather than running full-cluster scans.

Listing 1. Cassandra Table Schema Definition

```
CREATE KEYSPACE IF NOT EXISTS splunk_data
WITH replication = {'class': 'SimpleStrategy',
                    'replication_factor': 2};

CREATE TABLE IF NOT EXISTS splunk_data.logs (
    hostname text,
    timestamp timestamp,
    id uuid,
    process text,
    severity text,
    message text,
    PRIMARY KEY ((hostname), timestamp, id)
) WITH CLUSTERING ORDER BY (timestamp DESC, id ASC);
```

D. Deployment and Containerization

The microservices are defined using a single `docker-compose.yml` manifest. A shared bridge network (`splunk-net`) enables internal DNS resolution across services.

TABLE I
DOCKER COMPOSE SERVICE CONFIGURATION

Service	Base Image	Port	Purpose
`api-gateway`	Python 3.10 / aiohttp	8080	Request Routing
`cassandra`	Apache Cassandra 4.0	9042	Data Storage
`rabbitmq`	RabbitMQ 3-management	5672	Message Queue
`redis`	Redis 7-alpine	6379	Distributed Locks
`worker`	Python 3.10	–	Log Processing

III. TEST AND RESULT

A. Core Functionality and Performance

The system was evaluated using syslog batches ranging from 1,000 to 100,000 lines. Scatter-gather queries targeting hostnames, process daemons, severity levels, keyword search terms, and timestamps were executed to verify retrieval accuracy and response consistency across the dataset.

Introducing RabbitMQ buffered incoming traffic spikes effectively, maintaining stable API Gateway ingestion throughput at over 3,200 requests per second (RPS) without causing socket timeout errors or dropouts.

B. Fault Tolerance and Chaos Testing

The current implementation provides basic reliability through exception handling and persistent storage. The API Gateway catches processing errors and returns informative JSON responses without terminating the application. Cassandra ensures that ingested log entries remain stored until explicitly removed. Although the Worker Node only simulates background processing, it demonstrates a possible extension for future distributed processing.

IV. CONCLUSION

This Distributed "Mini-Splunk" Log Analytics Ecosystem implemented a containerized Mini-Splunk log analytics system that supports log ingestion, storage, querying, and deletion through RESTful services. The system consists of a Forwarder Client, an API Gateway, a Cassandra database, and a Worker Node simulation, demonstrating modular software design and persistent data management.

The implementation successfully achieved its primary objectives by providing structured log parsing, database storage, and flexible search capabilities within a containerized environment. While advanced distributed features such as

asynchronous messaging, clustered databases, and distributed fault tolerance are not included, the architecture provides a solid foundation for future enhancements involving scalable processing and distributed storage.

REFERENCES

[1] Common-OS, "Log Aggregation Pattern," *Commons-OS/patterns*, 2026. [Online]. Available: https://github.com/commons-os/patterns/blob/main/_patterns/log-aggregation-pattern.md

[2] A. Verma, "Telescope: Distributed Log Search Engine," GitHub Repository, 2026. [Online]. Available: https://github.com/Abhinnavverma/Telescope-Distributed-Log-Search-Engine

[3] K. Keswaran, "AI-Powered Observability and Log Analysis System," GitHub Repository, 2025. [Online]. Available: https://github.com/KeerthiKeswaran/AI-Powered-Observability-and-Log-Analysis-System

[4] Broadcom Inc., "Syslog Collector," *VMware Telco Cloud Service Assurance Documentation*, 2025. [Online]. Available: https://techdocs.broadcom.com/us/en/vmware-sde/telco-cloud/vmware-telco-cloud-service-assurance/2-4-0/telco-core/vmware-telco-cloud-service-assurance-configuration-guide1/configuring-administration-settings/managing-collectors-and-connectors/configuring-collector/syslog-collector.html

[5] Apache Software Foundation, "Apache Cassandra Documentation," 2023. [Online]. Available: https://cassandra.apache.org/doc/latest/